

Introduction to Computing: Tutorial 4

BSDS I Year, 2026–2027

September 24, 2026

1. Write Python functions `mode(L)` and `median(L)` that take a list `L` of n numbers as input, and return the mode and median of the numbers, respectively.

Recall that the *mode* is the element (or elements) that occurs most often in `L`, while the *median* is the middle element if the elements of `L` are listed in sorted order.

2. Write a Python function `logarithm(x, n)` that computes $\log_{10}(x)$ without using any library functions. Use trial and error, but ensure that, with every attempt, your ‘guesses’ get progressively closer to the desired value. See Q3 from Tutorial 1 for ideas.

3. Write a Python function `are_anagrams(s1, s2)` that checks whether the strings `s1` and `s2` are anagrams of each other, and returns `True` or `False` accordingly.

[Recall that `s1` and `s2` are anagrams if they correspond to the same multiset of letters, i.e., both contain the same letters, and the frequency of each letter is the same in both the strings.]

4. Write a Python function that takes a string `s` as input, and reverses each individual word within the string while maintaining the original word order. For example, if the input is “What a boring class”, your output should be “tahW a gnirob ssalc”.

Modify your program so that any word that starts with a capital letter also starts with a capital letter in the output. For the above example, your modified program should print `Tahw` as the first word, instead of `tahW`.

5. *Hangman* is a game played by two players. One of the players *A* thinks of a word (say w), consisting of m letters, and presents it to the other player *B*, with some/all letters missing. *B* tries to guess the word, one letter at a time. If the letter guessed by *B* occurs in w , *A* fills in **all** the positions in w where the letter occurs; otherwise, *B* loses a ‘life’. The process continues, until *B* correctly guesses the word, or runs out of lives.

Write a function `hangman(word, guesses, lives)` that takes two strings and an integer as parameters, displays the progress of the game, and finally prints whether

- *B* has correctly guessed the word (in this case, if there are any extra letters in `guesses`, they are to be ignored);
- *B* has run out of lives; or
- the game is incomplete.

For example, if the function is called as `hangman("lion", "euiaoblnxy", 6)`, it should print something like the following:

```
- - - -      6 lives remaining
- - - -      "e" guessed, 5 lives remaining
- - - -      "u" guessed, 4 lives remaining
_ i _ -      "i" guessed, 4 lives remaining
_ i _ -      "a" guessed, 3 lives remaining
_ i o _      "o" guessed, 3 lives remaining
_ i o _      "b" guessed, 2 lives remaining
l i o _      "l" guessed, 2 lives remaining
l i o n      "n" guessed, 2 lives remaining
Word correctly guessed.
```